

Kudde - User & Admin Manual

Kudde ("herd", in Afrikaans) is a small, self-hosted app for small-scale cattle recording. It tracks the animals in a herd and the handful of things that happen to them - a birth, a weighing, a treatment, a move between camps, a sale, a death - from one local server that any phone, tablet, or PC on the farm's own network can open.

This manual is for everyone who touches the app:

- **Field capture** - whoever records animals and events out at the herd, usually from a phone or tablet.
- **Farm admin / office staff** - whoever manages the master herd list, corrects records, and runs the server itself.

Table of Contents

1. [Overview & Concepts](#)
 2. [Server Setup](#)
 3. [Device Setup](#)
 4. [Field App - Recording the Herd](#)
 5. [Admin - Dashboard](#)
 6. [Admin - Herd](#)
 7. [Troubleshooting / FAQ - Annexe A: Data Field Reference](#)
-

1. Overview & Concepts

What Kudde is (and isn't)

Kudde is deliberately small. There is a proper international standard for recording livestock data - ICAR's Animal Data Exchange - and it defines around 75 resource and event types: breeding values, embryo flushing, carcass ultrasound, DNA results, formal conformation scoring, and more, built for stud and champion breeding operations. A small-scale farm needs a fraction of that: who an animal is, and the handful of things that happen to it. Kudde keeps the identity fields a stud registry would recognise (ear tag, breed, sex, parentage, birth date) and the event types that actually apply on a small farm (birth, weight, treatment, movement, death, sale) - trimmed down, not guessed from scratch.

There's no herdbook registration, no breed-code lookup, no formal scoring. Breed is free text. Parentage is recorded by tag, not by looking an animal up in this database - a sire or dam may have been sold, may have died, or may never have been entered here at all (a bought-in animal, a bull borrowed from a neighbour), so it has to be recordable even when it points at nothing Kudde knows about.

The two device roles

Every device opens one of two fixed web addresses, depending on what it's for:

Role	Address	Used by	What it does
Field	/field/	Whoever is out with the herd	Add animals, record events, browse the herd - works offline
Admin	/admin/	Farm office	Full herd list with editing, dashboard, complete event history, manual event entry

Unlike some setups, there's no one-time device-ID screen that decides this for you - a device is whichever screen you open on it, and it's fine to have several tabs or devices open on either screen at once. There's also no sign-in and no password: **anything on the same network that opens /admin/ can edit the herd**. That's a deliberate simplicity for a one-farm, one-admin tool, not an oversight - but it does mean the Admin address is worth keeping off any network you don't trust, and Kudde currently has no equivalent of restricting Admin to a separate, trusted network the way some larger farm systems do. Don't forward the server's port through your router to the public internet; that would expose an unauthenticated screen that can edit or delete herd records to anyone who finds the address.

Why field capture is offline-first

Cattle aren't always where the WiFi is. The Field app is built so that **recording an animal or an event never depends on having a connection** - it's saved on the device first, and quietly synced to the server in the background whenever a connection becomes available. You can keep working through a signal dropout without losing anything or needing to notice it happened.

Animal status, and what changes it

Every animal is **alive**, **dead**, or **sold**. Nobody has to remember to update this separately: recording a **death** or **sale** event against an animal moves it to that status automatically, the moment the event is saved - in the Field app or in Admin, online or (for Field) queued offline. There's no way to record a death or sale event without the animal's status following it.

The six event kinds

Kind	What it records	Extra fields used
Birth	The animal was born	-
Weight	A weighing	Value, in kg
Treatment	A vet treatment, dosing, etc.	-
Movement	A move between camps/paddocks	Location (the camp name)
Sale	The animal was sold	-
Death	The animal died	-

Every event also carries a date and a free-text note, whatever the kind. `value` (kg) only means anything on a weight event, and `location` only means anything on a movement event - the other kinds simply leave those blank.

The Field app's quick-action buttons cover weight, treatment, movement, sale, and death - the events you'd realistically capture while standing next to the animal. There's no field quick-action for **birth**, since a newly recorded animal's birth date is normally set directly on the **Add animal** form instead. A birth event is there for the rarer case where you want to log the birth itself as a dated event with its own note - it is available from **Admin - Herd**'s event form.

"Needs weighing"

An alive animal with no weight event recorded in the last **182 days** (roughly six months) shows up as needing weighing, on both the Field app's summary and the Admin dashboard. Long enough that weighing every muster or two doesn't nag you; short enough to actually catch an animal that's fallen through the cracks. This is purely informational - nothing stops you from ignoring it - but it's the one thing in Kudde that proactively tells you an animal hasn't been looked at in a while.

2. Server Setup

Kudde is a single Python service (FastAPI) that serves its own frontend and stores everything in a local SQLite database file - there's no separate database server, cloud account, or build step. It runs on one ordinary computer (the "server"), and every other device just opens a web address pointing at that computer; nothing needs to be installed on those other devices beyond a browser.

Prerequisites

- A Windows, Mac, or Linux computer that stays switched on and connected to the network while the app is in use. It doesn't need to be powerful - it's one lightweight Python process and a small SQLite database, not a heavy computation.
- Python 3.9 or newer (the automated Windows installer gets you 3.11 if nothing suitable is already present).
- Network access from every device that needs to reach the app - the same Wi-Fi/LAN the farm already uses.
- The project folder, copied onto that computer (via `git clone`, a zip file, or a USB drive).

Don't put the project folder inside a synced cloud drive - Google Drive, OneDrive, Dropbox, iCloud. The sync client keeps file handles open, which can block updates or stop a folder from deleting, and syncing a live SQLite database file while it's being written is a known way to corrupt it. Back a finished backup copy up to the cloud by all means; just don't run the app from a synced folder.

Quick setup on Windows: the automated installer

The project folder includes `install.bat`, which does everything described below in one step and is safe to double-click again later if something needs redoing.

1. Get the project folder onto the PC.
 2. Double-click `install.bat`.
 3. If Windows shows a blue **"Windows protected your PC"** screen, click **"More info"**, then **"Run anyway"** - normal for any script that isn't from a large, registered publisher.
 4. If a **User Account Control** prompt appears, click **"Yes"** - administrator rights are needed to configure the firewall and register the auto-start task.
 5. Wait for it to finish. It will: - Install Python 3.11 if no compatible 64-bit Python 3.9+ is already on the PC.
- Create a virtual environment in `backend\.venv` and install the app's dependencies into it.

- Write a `start_server.bat` launcher.
 - Open port **8010** through Windows Firewall (a different port from Boord's 8000, so both can run on the same office PC without conflicting).
 - Register a Scheduled Task named exactly "**Kudde Server**" that starts the server automatically at boot, running as SYSTEM - no one needs to be logged in for it to start.
 - Start the server immediately and poll `http://localhost:8010/field/` for up to 20 seconds to confirm it actually came up, rather than just assuming it did.
 - Print the addresses to use, both on this PC and from other devices on the network.
1. Note what it prints at the end: **there is no sign-in**. Both `/field/` and `/admin/` open straight up for anyone on the same network - see [the security note in chapter 1](#).

Setting up by hand (Windows, Mac, or Linux)

Useful if the installer fails partway, or on Mac/Linux where there's no automated script yet. From the `backend/` folder:

Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8010
```

Mac/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8010
```

- `--host 0.0.0.0` makes the server reachable from other devices on the network, not just this machine.
- `8010` is the port every other step in this manual assumes; any free port works, but stick with one number once devices are configured against it.
- Find this machine's network address with `ipconfig` (Windows, look for "IPv4 Address") or `ifconfig / ip addr` (Mac/Linux, look for `inet` under the active adapter), so other devices can reach `http://<that-address>:8010/`.
- On Windows, allow the port through Windows Firewall if it isn't prompted automatically (Windows Security → Firewall & network protection → Advanced settings → Inbound Rules → New Rule → Port → TCP 8010 → Allow).

- To keep it running after closing the terminal and restart automatically after a reboot, use your platform's standard approach for a long-running service (a Scheduled Task on Windows - see below - a `launchd` agent on Mac, or a `systemd` service on Linux) pointed at the same `uvicorn` command.

What happens on first startup

The very first time the server runs, it creates `data\kudde.db` and brings the database schema up to date automatically (via Alembic migrations, run from `main.py`'s startup step) - there is nothing to run by hand for this, on a fresh install or after pulling an update that changes the schema. **No demo or sample data is seeded** - the herd list starts empty, ready for real animals.

Updating to a newer version of the code

Kudde doesn't yet have a signed-release update mechanism (something worth adding once it's running on real farms and a bad update could mean losing someone's herd records). For now, updating means:

1. Stop the server (see below).
2. Pull the new code (`git pull` , or replace the folder's contents with the new version, keeping `data\` untouched).
3. If `backend/requirements.txt` changed, re-run the installer or `pip install -r requirements.txt` inside `backend\.venv` to pick up any new dependency.
4. Restart the server - schema migrations run automatically the moment it starts back up.

Each device's browser tab or installed app also needs a full close-and-reopen after an update to pick up new frontend code - see [Confirming a device picked up an update](#) in chapter 3.

Stopping, starting, and restarting the server

If set up via `install.bat` , the server runs as a Scheduled Task named "**Kudde Server**", as SYSTEM, with no visible window.

Using Task Scheduler: open **Task Scheduler**, find "**Kudde Server**" in the Library, right-click → **End** to stop it, or **Run** to start it. To restart, do **End** then **Run**.

Using Command Prompt (as administrator):

```
schtasks /end /tn "Kudde Server"
schtasks /run /tn "Kudde Server"
```

Simplest restart: since the task launches automatically on every startup anyway, just restarting the PC has the same effect.

If you need it definitely stopped - before copying the database, for example - use

`stop_server.ps1` rather than End on its own:

```
powershell -NoProfile -ExecutionPolicy Bypass -File stop_server.ps1
```

Ending the Scheduled Task stops the launcher, but the server process it spawned is separate and can survive that, still holding port 8010 and the database open. This script ends the task, then waits until port 8010 is actually free, and reports what's still holding it if it never comes free. It only ever stops processes running from Kudde's own `backend\.venv` - if something else on the PC has taken port 8010, it says so and refuses to kill it.

Checking it's running: browse to `http://localhost:8010/field/` on the server PC - if the Field app loads, it's up.

Backing up your data

Everything Kudde knows lives in one file: `data\kudde.db`. There is currently no automatic backup - that is a real limitation, not a deliberate safety net waiting behind the scenes, and it's on you to protect a season's worth of records. Stop the server first (see above) so nothing is mid-write, then copy `data\kudde.db` somewhere safe (a USB drive, a cloud drive - copying a finished file to a sync folder is fine, even though running the app from one isn't). Do this regularly, and definitely before any update.

Removing Kudde from a PC

Double-click `uninstall.bat`. It stops and unregisters the auto-start Scheduled Task, removes the firewall rule, deletes the generated `start_server.bat`, and removes the Python virtual environment. It prints what's still on the machine afterward.

It does not delete `data\`, and it does not delete the project folder. That folder holds the herd database - every animal and event recorded - and reinstalling doesn't bring it back. Deleting it stays a deliberate act, so the uninstaller prints the commands rather than running them:

```
xcopy /E /I "C:\Kudde\data" "%USERPROFILE%\Desktop\kudde-data-backup"  
rmdir /s /q "C:\Kudde"
```

Copy first, and run the second command from a folder outside `C:\Kudde` - a Command Prompt sitting inside it will block its own delete.

3. Device Setup

There's no device-registration step. A device is whichever address you open on it:

- **Field:** `http://<server-address>:8010/field/`
- **Admin:** `http://<server-address>:8010/admin/`

Bookmark the right one on each device, or better, install it as an app icon.

Installing as an app icon (optional, recommended)

Both Field and Admin are small installable apps (PWAs) with their own name and navy icon, so a device can show "Kudde Field" or "Kudde Admin" on its home screen instead of a browser tab or bookmark. This isn't required - the browser bookmark works fine on its own - but it makes the right screen one tap away.

- **Android (Chrome):** tap the three-dot menu → **"Add to Home screen"** (or use the install banner/icon in the address bar if offered automatically).
- **iPhone/iPad (Safari):** tap the **Share** icon → **"Add to Home Screen"**.
- **Windows/Mac (Chrome or Edge):** look for an install icon (a monitor with a down-arrow) at the right end of the address bar, or the three-dot menu → **"Install [app name]..."**.

Install from the address you actually intend to use (`.../field/` or `.../admin/`) so the shortcut opens the right screen with the right name and icon.

Confirming a device picked up an update

Each screen's header shows its version number (e.g. `v0.1`) next to the sync status pill. Because the app is a PWA with a service worker, an installed icon can keep showing the old frontend after the server itself has been updated, until it's fully closed and reopened. If a device looks like it hasn't picked up a recent change, check the version number first; if it's behind, close the app completely (not just switch away from it) and reopen it. A leftover home-screen icon from an old install can also point at stale cached data - if renaming or re-icon-ing an install, remove the old icon and clear the site's data before reinstalling.

4. Field App - Recording the Herd

Header

Shows "Kudde / Field", the sync status pill, and the app version. The sync pill reads:

Pill text	Meaning
Online	Connected, nothing waiting to sync
Syncing N...	Connected, N queued items being sent now
Offline	Can't reach the server right now
Offline - N pending	Can't reach the server; N items are queued and will send once it's back

The offline banner

A banner reading *"Offline - changes are saved on this device and will sync later"* appears whenever the app can't reach the server. It's informational, not a block - every screen still works.

Pull down to refresh

Installed as a home-screen app, there's no browser reload button, so pull down from the top of the list to re-fetch the herd and dashboard summary (and trigger a sync attempt) by hand. This is a touch gesture only.

Searching and adding an animal

The search box at the top filters the herd list live, by tag (matches anywhere) or name. **Add animal** opens a form for:

- **Tag number** (required) - the ear tag number, and the animal's permanent identity. Kudde refuses a tag already in use.
- **Name** (optional)
- **Sex** (required) - Female or Male
- **Breed** (optional, free text)
- **Birth date** (optional)
- **Sire tag / Dam tag** (optional) - the parent's ear tag, if known. These don't have to exist in Kudde's own records; a bought-in animal or a bull borrowed from a neighbour can still be recorded as a parent.

New animals start **Alive**.

The herd summary and list

Below the search box, two counts: **Alive**, and **Need weighing** (see "[Needs weighing](#)" in chapter 1). The list below shows every cached animal, sorted by tag, each with a status pill (Alive/Dead/ Sold). Tap any animal to open its detail.

Animal detail and recording an event

Tapping an animal opens its tag, name/breed/sex, status pill, and five quick-action buttons:

Weight · Treatment · Move · Sale · Death

Each opens a small form: a date (defaults to today), a value in kg (weight only), a camp/location (movement only), and a free-text note. Saving:

- Sends the event to the server immediately if reachable.
- If not, saves it to this device's outbox and shows *"Saved on this device - will sync when online"* instead.
- For a **Sale** or **Death** event, flips the animal's status right away on this device (the same rule the server applies), so the list and the detail card are correct immediately, connected or not.

Below the action buttons, **History** lists every recorded event for that animal, newest first - unavailable while offline, since it isn't cached locally the way the herd list is.

Offline-first capture, in detail

Adding an animal or recording an event always succeeds locally first. Behind the scenes:

- The herd list is cached on the device (IndexedDB), so it's browsable offline - though each animal's event *history* is fetched live and isn't available offline.
 - Anything that couldn't reach the server goes into an on-device outbox and is retried automatically: every 20 seconds, the moment the browser reports it's back online, and on every pull-to-refresh.
 - If the server later rejects a queued item outright (most commonly, a tag that got created from another device first and synced sooner), that one entry is dropped rather than blocking every entry queued behind it. **Two devices creating the same new tag while both offline is the one case Kudde can't reconcile for you** - whichever syncs first wins, and the second is silently lost. If more than one device might be adding new animals, agree tag ranges in advance, or check the Herd tab in Admin before working somewhere with no signal.
-

5. Admin - Dashboard

The Admin app's landing tab. If the server can't be reached, the sync pill in the header reads "**Offline - the server on this network can't be reached**" and the tab shows whatever it last loaded - there's no offline cache here the way the Field app has one, since Admin is meant to be run from the office, on the same network as the server.

Herd counts

Three tiles: **Alive**, **Dead**, **Sold** - a live count straight from the database.

Needs weighing

Every alive animal with no weight event in the last 182 days (see [chapter 1](#)), with a **View** link that jumps to the Herd tab and opens that animal's edit screen directly.

Recent activity

The 10 most recent events across the whole herd, newest first - whatever kind, from whichever device recorded them.

6. Admin - Herd

The full, editable herd list.

Searching and filtering

A search box (tag or name) and a status dropdown (All / Alive / Dead / Sold) filter the table live. **Add animal** opens the same fields as the Field app's Add animal form.

The herd table

Columns: Tag, Name, Breed, Sex, Birth date, Status. Click any row to open that animal.

Editing an animal

Opens every field the Add form has, plus **Status** (Alive/Dead/Sold) - editable directly here, in addition to being set automatically by a sale or death event. The **tag number can't be changed** once an animal is created; it's the identity everything else (events, parentage links from other animals) is recorded against.

History and recording an event

Below the fields, when editing an existing animal: its full event history, and a compact form to record a new one directly - date, kind (**Weight, Treatment, Movement, Birth, Sale, Death** - including Birth, unlike the Field app's quick actions), value in kg (weight only), location (movement only), and a note. Saving refreshes the history list, the herd table, the dashboard, and - if the event was a sale or death - the status pill shown at the top of this same modal, without needing to reopen it.

7. Troubleshooting / FAQ

Admin says "Offline - the server on this network can't be reached". The server PC is off, asleep, or unreachable from this device's network. Confirm the server PC is on and the app is running (`http://localhost:8010/field/` loads on the server PC itself), that this device is on the same network, and that port 8010 is allowed through the server's firewall.

Field says "Offline - N pending". Normal while out of signal range - those N animals/events are saved on this device and will send automatically once it reconnects (or pull down to refresh to retry immediately). Nothing is lost by staying offline; it's only lost if the device itself is lost or its browser data is cleared before it syncs.

"Tag ... is already in use". Tags are permanent identities - even a dead or sold animal keeps its tag forever, so it can't be reused for a new one. Search for it in Admin → Herd with the status filter cleared to find the existing record.

Two devices added the same new tag while both were offline. See [Offline-first capture, in detail](#) - this is a known limitation, not a bug: whichever syncs first keeps the tag, and the other capture is dropped. Agree on tag ranges up front if more than one device might be adding new animals away from a connection.

A device looks like it hasn't picked up a recent update. Check the version number in the header first (see [chapter 3](#)); if it's behind, fully close and reopen the app rather than just switching away and back.

The server won't start, or a browser says the connection was refused. Something may already be listening on port 8010 - most often a leftover `python.exe` from a manual `uvicorn` run left in Task Manager after a Scheduled Task restart. End it, then start the "Kudde Server" task again, or use `stop_server.ps1` first to confirm the port is genuinely free.

The "Kudde Server" scheduled task shows as ran/ready, but nothing answers. Check the task's History tab in Task Scheduler for an error, or run `start_server.bat` by hand in a visible Command Prompt window - errors that a headless Scheduled Task swallows are printed there.

How do I get my data off this PC, or back it up? Everything is in `data\kudde.db`. See [Backing up your data](#) in chapter 2 - there's currently no automatic backup, so this is a manual habit worth keeping.

Can I reach Kudde from off the farm? Not out of the box - Kudde doesn't include or require a VPN or remote-access setup. If you need that, you'd need to set one up yourself (e.g. Tailscale) pointed at this server. Whatever you use, **never** forward port 8010 directly through your router to the public internet: Admin has no sign-in, so doing that would let anyone on the internet who finds the address edit or delete your herd records.

Annexe A: Data Field Reference

Animal

Field	Type	Notes
<code>id</code>	integer	Internal id, assigned by the database
<code>tag</code>	text	The ear tag number. Unique, and the animal's permanent identity
<code>name</code>	text	Optional
<code>breed</code>	text	Optional, free text - no breed-code lookup
<code>sex</code>	<code>male</code> <code>female</code>	Required
<code>birth_date</code>	date	Optional
<code>sire_tag</code>	text	Optional. The father's ear tag - not required to exist as an <code>Animal</code> in this database
<code>dam_tag</code>	text	Optional. The mother's ear tag - same rule as <code>sire_tag</code>
<code>status</code>	<code>alive</code> <code>dead</code> <code>sold</code>	Set to <code>alive</code> on creation; changes automatically when a <code>death</code> or <code>sale</code> event is recorded, or can be set directly in Admin
<code>created_at</code>	timestamp	Set once, on creation
<code>updated_at</code>	timestamp	Updated on every change

Event

Field	Type	Notes
<code>id</code>	integer	Internal id
<code>animal_id</code>	integer	The animal this event belongs to
<code>kind</code>	<code>birth</code> <code>weight</code> <code>treatment</code> <code>movement</code> <code>death</code> <code>sale</code>	
<code>event_date</code>	date	Required
<code>value</code>	number	Weight in kg - only meaningful on a <code>weight</code> event
<code>note</code>	text	Optional, free text, any kind

Field	Type	Notes
location	text	The camp/paddock name - only meaningful on a movement event
created_at	timestamp	Set once, on creation

death and sale events additionally set the parent Animal.status to dead or sold respectively, the moment they're recorded.